

UNIVERSIDAD DE LAS PALMAS DE GRAN CANARIA
Escuela de Ingeniería Informática

Práctica 4:
Correo Electrónico Seguro S/MIME

Asignatura: Seguridad de la Información

Curso: 4º de Grado en Ingeniería Informática

Autor:

Nicolás Rey Alonso

`nicolas.rey101@alu.ulpgc.es`

Fecha: Febrero 2026

Índice general

1. Introducción	3
1.1. Objetivo de la práctica	3
1.2. Conceptos fundamentales	3
1.2.1. Certificados digitales X.509	3
1.2.2. S/MIME: Correo electrónico seguro	4
1.2.3. Estructura de un mensaje cifrado y firmado	4
1.2.4. Formatos de ficheros	5
1.3. Herramientas utilizadas	5
2. Creación del Certificado Personal	6
2.1. Descripción teórica	6
2.2. Paso a) – Descargar el certificado raíz	6
2.3. Paso b) – Crear la clave del certificado personal	7
2.4. Paso c) – Crear el CSR (Certificate Signing Request)	7
2.5. Paso d) – Firmar el CSR con el certificado raíz	8
2.6. Paso e) – Crear el fichero PKCS#12	8
2.7. Ficheros resultantes	9
3. Instalación de Certificados en Thunderbird e Intercambio de Correo S/MIME	10
3.1. Instalación del certificado raíz	10
3.2. Instalación del certificado personal	11
3.3. Configuración de la cuenta de correo	12
3.4. Intercambio de certificados con la compañera	12

3.4.1. Envío de mensaje firmado	13
3.4.2. Certificado de la compañera en Thunderbird	14
3.5. Envío de mensaje cifrado y firmado	15
3.6. Recepción del mensaje cifrado y firmado de la compañera	15
4. Decodificación S/MIME desde Línea de Comandos	17
4.1. Descripción del proceso	17
4.2. Ficheros necesarios	17
4.3. Paso 1 – Descifrado del mensaje (SMIME.P7M → SMIME.P7S)	18
4.4. Paso 2 – Verificación de la firma (SMIME.P7S → texto plano)	19
4.5. Resultado del proceso	20
4.6. Resumen del flujo completo	21
5. Conclusiones	22
5.1. Resumen de aprendizajes	22
5.2. Conclusiones sobre la práctica	22
5.3. Problemas encontrados y soluciones	23
A. Comandos OpenSSL Utilizados	25
A.1. Creación de certificados	25
A.2. Operaciones S/MIME	25

Capítulo 1

Introducción

1.1. Objetivo de la práctica

El objetivo principal de esta práctica es configurar y utilizar **correo electrónico seguro** mediante el estándar **S/MIME** (*Secure/Multipurpose Internet Mail Extensions*), incluyendo:

- Creación de un certificado digital personal X.509 firmado por una CA raíz
- Instalación de certificados en el cliente de correo Mozilla Thunderbird
- Intercambio de mensajes firmados y cifrados con otro compañero
- Decodificación de mensajes S/MIME cifrados y firmados desde línea de comandos con `openssl smime`

1.2. Conceptos fundamentales

1.2.1. Certificados digitales X.509

Un **certificado digital** vincula la identidad de un sujeto (nombre, correo electrónico, organización) con su **clave pública**. Esta vinculación está firmada digitalmente por una **Autoridad de Certificación (CA)**, lo que permite a cualquier parte que confíe en esa CA verificar la autenticidad del certificado.

La estructura básica de un certificado X.509 incluye:

- **Sujeto (Subject):** Identidad del titular (CN, O, OU, C, Email)
- **Emisor (Issuer):** CA que firma el certificado

- **Clave pública:** La clave pública del titular
- **Periodo de validez:** Fechas de inicio y expiración
- **Firma digital:** Firma de la CA sobre todos los campos anteriores

1.2.2. S/MIME: Correo electrónico seguro

S/MIME utiliza criptografía de clave pública y certificados X.509 para proporcionar tres servicios de seguridad al correo electrónico:

Cuadro 1.1: Servicios de seguridad de S/MIME

Servicio	Mecanismo	Descripción
Confidencialidad	Cifrado	Solo el destinatario puede leer el mensaje (cifrado con su clave pública)
Autenticación	Firma digital	Se verifica la identidad del emisor
Integridad	Firma digital	Se detecta si el mensaje fue alterado

1.2.3. Estructura de un mensaje cifrado y firmado

Cuando un mensaje S/MIME es tanto firmado como cifrado, se crea una estructura de **dobles envoltente**:

1. El emisor **firma** el mensaje con su clave privada → genera estructura `signed-data` (SMIME.P7S)
2. El resultado se **cifra** con la clave pública del destinatario → genera estructura `enveloped-data` (SMIME.P7M)

Para decodificarlo, el receptor realiza el proceso inverso:

1. **Descifra** con su clave privada (SMIME.P7M → SMIME.P7S)
2. **Verifica la firma** con el certificado del emisor (SMIME.P7S → texto plano)

1.2.4. Formatos de ficheros

Cuadro 1.2: Formatos de ficheros utilizados en esta práctica

Extensión	Formato	Contenido
.crt	PEM (Base64)	Certificado X.509 (solo parte pública)
.key	PEM (Base64)	Clave privada (puede estar cifrada con contraseña)
.csr	PEM (Base64)	Solicitud de firma de certificado
.p12	PKCS#12 (binario)	Certificado + clave privada empaquetados
.eml	Texto	Mensaje de correo completo (cabeceras + cuerpo MIME)
.p7m	PKCS#7 (binario)	Mensaje cifrado S/MIME (enveloped-data)
.p7s	PKCS#7 (binario)	Mensaje firmado S/MIME (signed-data)

1.3. Herramientas utilizadas

- **OpenSSL 3.x:** Herramienta de criptografía en línea de comandos
- Comandos principales: `genpkey`, `req`, `x509`, `pkcs12`, `smime`
- **Mozilla Thunderbird:** Cliente de correo electrónico con soporte S/MIME

Capítulo 2

Creación del Certificado Personal

2.1. Descripción teórica

Un **certificado digital X.509** es un documento electrónico que vincula una identidad (nombre, correo electrónico, organización) con una clave pública. Esta vinculación está firmada por una **Autoridad de Certificación (CA)**, lo que permite a terceros confiar en que esa clave pública pertenece realmente a la identidad indicada.

El proceso de creación de un certificado personal implica los siguientes pasos:

1. **Generar un par de claves** (pública y privada) para el usuario.
2. **Crear una solicitud de firma (CSR)**: documento que contiene la clave pública y los datos del solicitante.
3. **Firmar el CSR** con el certificado y clave de la CA raíz, generando el certificado final (.crt).
4. **Empaquetar en PKCS#12**: agrupar certificado y clave privada en un único fichero protegido (.p12) para su importación en aplicaciones.

En esta práctica se utiliza un certificado raíz proporcionado por la asignatura (`certificadoRaiz.crt` y `certificadoRaiz.key`, protegido con la contraseña `SEGURIDAD`) como CA que firma nuestro certificado personal.

2.2. Paso a) – Descargar el certificado raíz

Se descargan los ficheros `certificadoRaiz.crt` (parte pública) y `certificadoRaiz.key` (clave privada, protegida con contraseña `SEGURIDAD`) desde la página de la asignatura y se co-

locan en el directorio de trabajo.

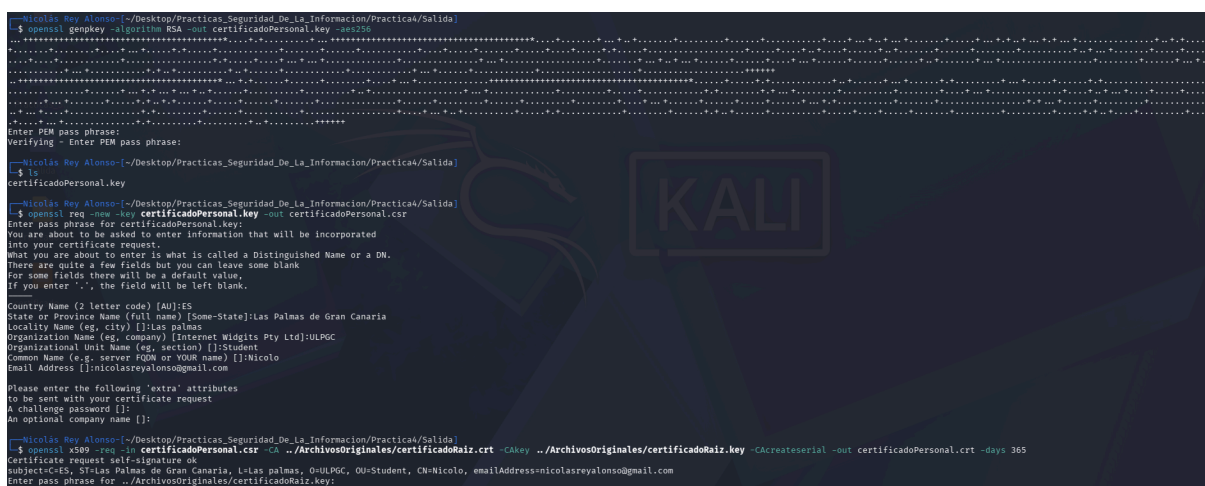
2.3. Paso b) – Crear la clave del certificado personal

Se genera una clave privada RSA protegida con cifrado AES-256:

Listing 2.1: Generación de la clave privada del certificado personal

```
openssl genpkey -algorithm RSA -out certificadoPersonal.key -aes256
```

OpenSSL solicita una contraseña para proteger la clave privada. Esta contraseña será necesaria cada vez que se utilice la clave (por ejemplo, al firmar el CSR o al descifrar mensajes).



```
Nicolas Rey Alonso:~/Desktop/Practicas_Seguridad_De_La_Informacion/Practica4/Salida
$ openssl genpkey -algorithm RSA -out certificadoPersonal.key -aes256
.....
Enter PEM pass phrase:
Verifying - Enter PEM pass phrase:
$ ls
certificadoPersonal.key
$ openssl req -new -key certificadoPersonal.key -out certificadoPersonal.csr
Enter pass phrase for certificadoPersonal.key:
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:ES
State or Province Name (full name) [Some-State]:Las Palmas de Gran Canaria
Locality Name (eg, city) []:Las Palmas
Organization Name (eg, company) [Internet Widgits Pty Ltd]:ULPGC
Organizational Unit Name (eg, section) []:Student
Common Name (e.g. server FQDN or YOUR name) []:Nicolas
Email Address []:nicolasreyalons@gmail.com

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:
An optional company name []:

Nicolas Rey Alonso:~/Desktop/Practicas_Seguridad_De_La_Informacion/Practica4/Salida
$ openssl x509 -req -in certificadoPersonal.csr -CA ../ArchivosOriginales/certificadoRaiz.crt -CAkey ../ArchivosOriginales/certificadoRaiz.key -CAcreateserial -out certificadoPersonal.crt -days 365
Certificate request self-signature ok
subject=C=ES, ST=Las Palmas de Gran Canaria, L=Las Palmas, O=ULPGC, OU=Student, CN=Nicolas, emailAddress=nicolasreyalons@gmail.com
Enter pass phrase for ../ArchivosOriginales/certificadoRaiz.key:
```

Figura 2.1: Generación de la clave personal y creación del CSR en terminal

2.4. Paso c) – Crear el CSR (Certificate Signing Request)

El CSR incluye la clave pública y los datos identificativos del solicitante (país, organización, nombre, correo):

Listing 2.2: Creación del CSR

```
openssl req -new -key certificadoPersonal.key -out
↪ certificadoPersonal.csr
```

Durante el proceso se introducen los campos del certificado:

- **C** (Country): ES
- **O** (Organization): ULPGC – Seguridad de la Información

- **OU** (Organizational Unit): EII
- **CN** (Common Name): Nicolás Rey Alonso
- **Email**: dirección de correo personal

2.5. Paso d) – Firmar el CSR con el certificado raíz

Se firma el CSR utilizando el certificado raíz y su clave privada, generando nuestro certificado personal con validez de 365 días:

Listing 2.3: Firma del CSR con el certificado raíz

```
1 openssl x509 -req -in certificadoPersonal.csr \  
2   -CA certificadoRaiz.crt -CAkey certificadoRaiz.key \  
3   -CAcreateserial -out certificadoPersonal.crt -days 365
```

Se introduce la contraseña **SEGURIDAD** que protege `certificadoRaiz.key`. El resultado es el fichero `certificadoPersonal.crt` que contiene nuestra clave pública firmada por la CA raíz.



```
—Nicolás Rey Alonso— /Desktop/Practicas Seguridad De La Informacion/Practicas/Solida  
$ openssl pkcs12 -export -in certificadoPersonal.crt -inkey certificadoPersonal.key -out ./certificadoPersonal.p12 -name "Certificado Personal"  
Enter pass phrase for certificadoPersonal.key:  
Enter Export Password:  
Verifying - Enter Export Password:
```

Figura 2.2: Firma del CSR y creación del fichero PKCS#12

2.6. Paso e) – Crear el fichero PKCS#12

El formato PKCS#12 (.p12) empaqueta el certificado personal y la clave privada en un único fichero protegido con contraseña, adecuado para importar en clientes de correo como Thunderbird:

Listing 2.4: Creación del fichero PKCS#12

```
1 openssl pkcs12 -export -in certificadoPersonal.crt \  
2   -inkey certificadoPersonal.key \  
3   -out certificadoPersonal.p12 -name "Certificado Personal"
```

El comando solicita:

1. La contraseña de `certificadoPersonal.key` (la que creamos en el paso b).
2. Una nueva contraseña de exportación para proteger el fichero `.p12`.

2.7. Ficheros resultantes

Tras completar el proceso, se dispone de los siguientes ficheros:

Cuadro 2.1: Ficheros generados en la creación del certificado

Fichero	Formato	Contenido
certificadoPersonal.key	PEM	Clave privada RSA cifrada con AES-256
certificadoPersonal.crt	PEM	Certificado personal (clave pública firmada por la CA)
certificadoPersonal.p12	PKCS#12	Clave privada + certificado en un solo fichero

El fichero `certificadoPersonal.csr` se elimina tras la firma, ya que no es necesario conservarlo.

Capítulo 3

Instalación de Certificados en Thunderbird e Intercambio de Correo S/MIME

3.1. Instalación del certificado raíz

El primer paso es importar la parte pública del certificado raíz (`certificadoRaiz.crt`) en Mozilla Thunderbird para que el cliente de correo confíe en los certificados firmados por esta CA.

En Thunderbird, se accede a **Herramientas** → **Opciones** → **Avanzado** → **Administrar certificados** (o **Ajustes** → **Privacidad y seguridad** → **Gestionar certificados**).

Al importar el certificado raíz se le asigna confianza para *identificar direcciones de correo electrónico*. El certificado aparece en la pestaña **Autoridades**, bajo la organización “ULPGC – Seguridad de la Información”.

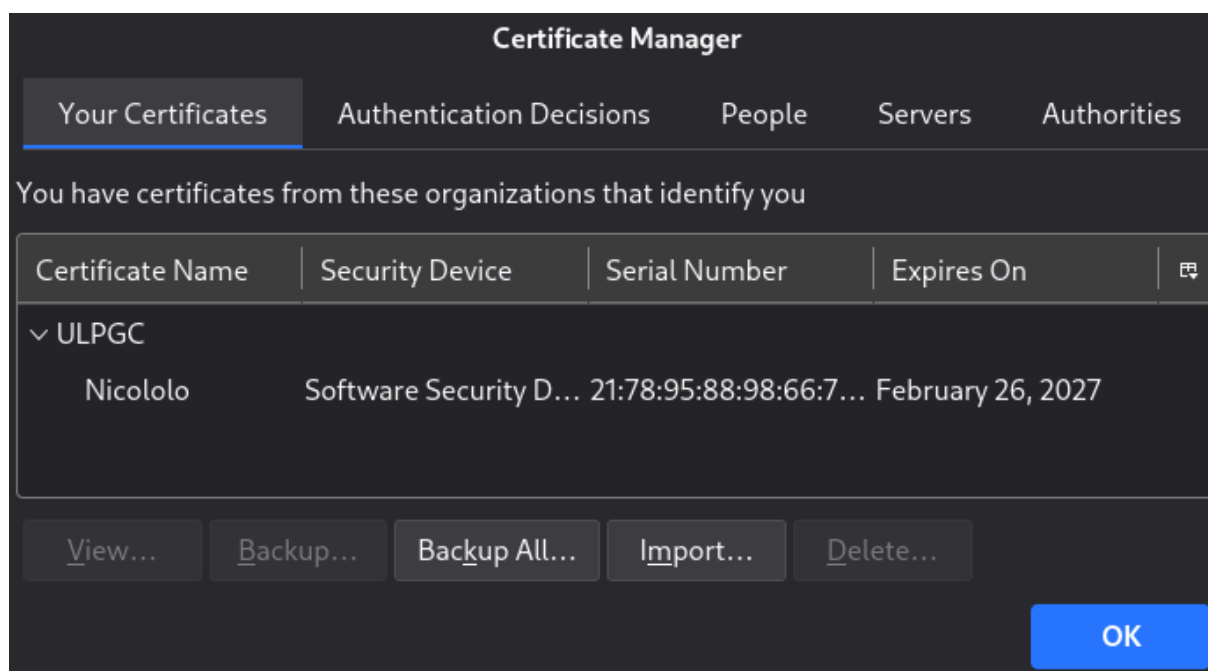


Figura 3.1: Importación del certificado raíz en Thunderbird (almacén de Autoridades)

3.2. Instalación del certificado personal

A continuación se importa el fichero `certificadoPersonal.p12` que contiene tanto la clave privada como el certificado personal. Thunderbird solicita la contraseña de protección del `.p12` que establecimos anteriormente.

Tras la importación, nuestro certificado aparece en la pestaña **Sus certificados**.

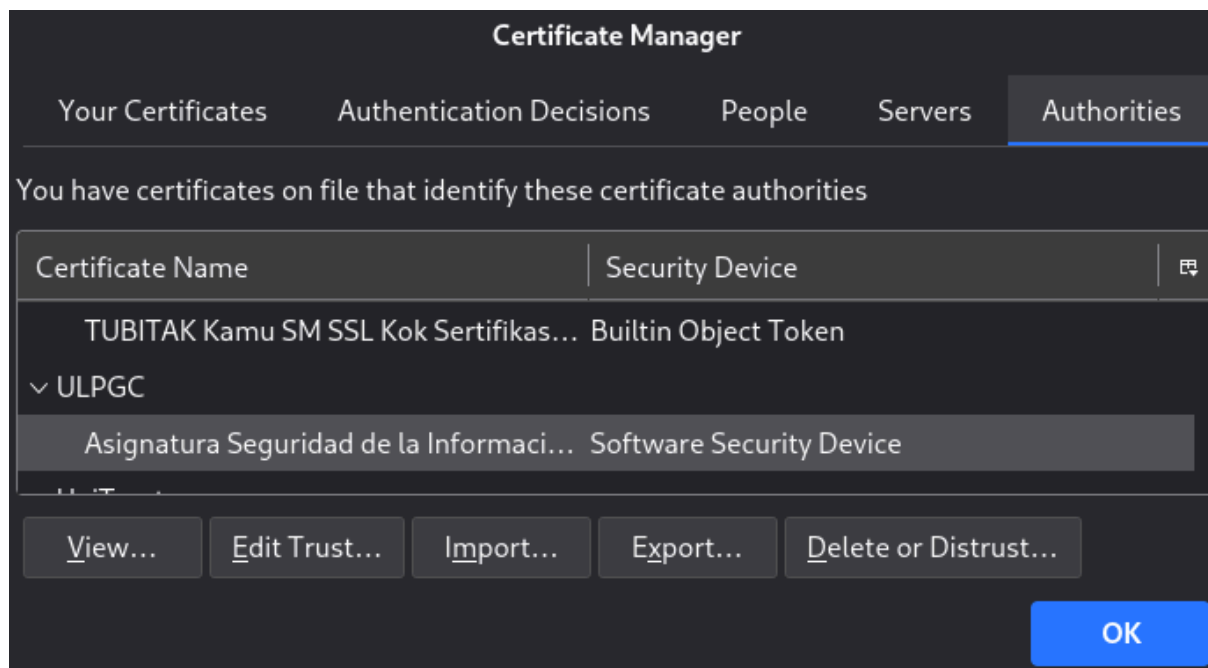


Figura 3.2: Certificado personal importado en Thunderbird (pestaña “Sus certificados”)

3.3. Configuración de la cuenta de correo

Para que Thunderbird utilice nuestro certificado al firmar y cifrar mensajes, se configura la cuenta:

1. Acceder a **Herramientas** → **Configuración de cuenta** → **Cifrado de extremo a extremo** (o **Seguridad** en versiones anteriores).
2. En la sección de **S/MIME**, pulsar *Seleccionar* tanto para **Firmado digital** como para **Cifrado**.
3. Seleccionar nuestro certificado personal en ambos casos.
4. **Importante:** No marcar las opciones de firmar y cifrar siempre por defecto; es preferible decidir para cada mensaje individualmente.

3.4. Intercambio de certificados con la compañera

Para poder enviar correo cifrado a un destinatario, es necesario disponer previamente de su certificado (clave pública). El procedimiento de intercambio es:

1. **Enviar un mensaje solo firmado** a la compañera (Wafa Azdad Triki). Al firmar, se adjunta automáticamente nuestro certificado público.

2. La compañera recibe el mensaje firmado y Thunderbird almacena nuestro certificado en el almacén de **Otras personas**.
3. La compañera nos envía un mensaje firmado a nosotros o con su certificado adjunto, como en este caso, de modo que recibimos su certificado.
4. Verificar que el certificado del compañero aparece en **Otras personas** en el administrador de certificados.

3.4.1. Envío de mensaje firmado

Se compone un mensaje con el asunto “Nicolás Rey Alonso” dirigido a la compañera, activando la opción de **firma digital** (sin cifrado, ya que aún no tenemos su certificado):

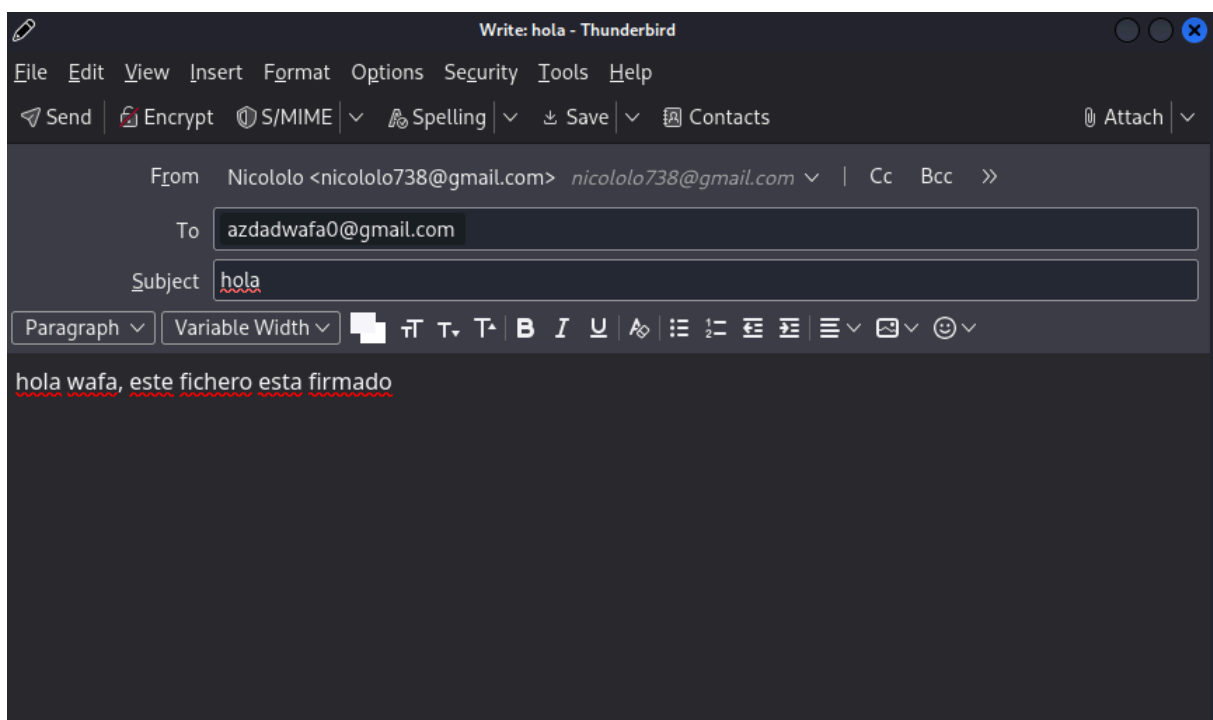


Figura 3.3: Composición de mensaje firmado digitalmente

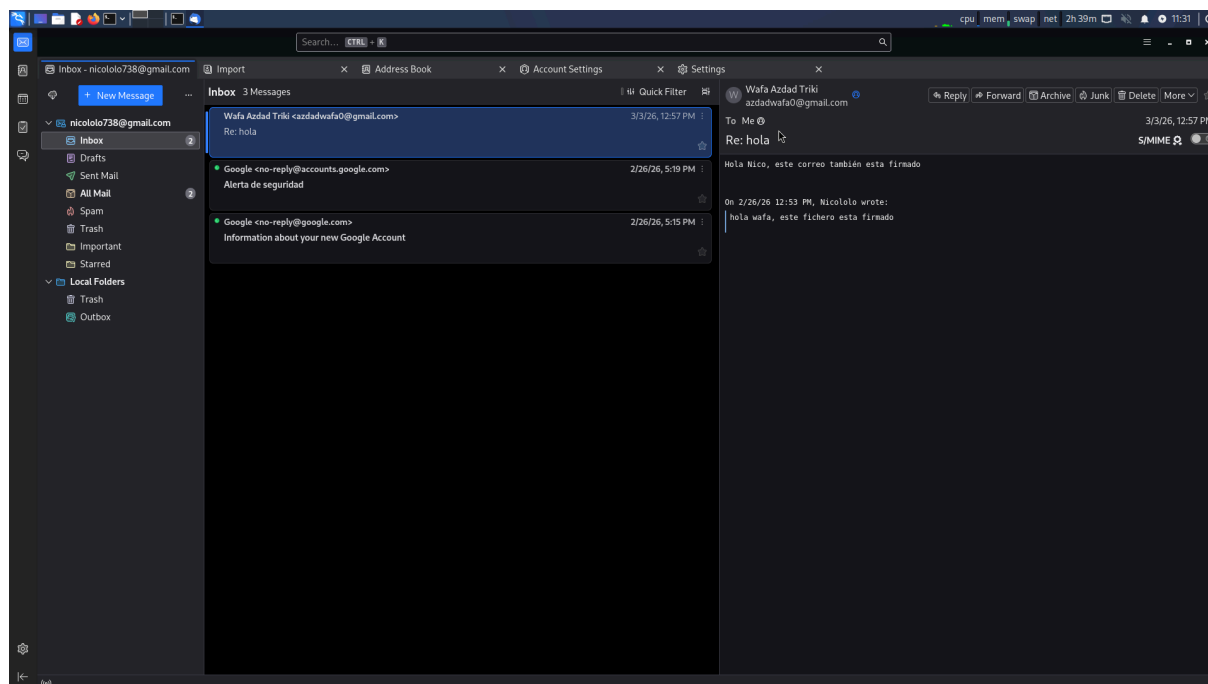


Figura 3.4: Mensaje firmado enviado correctamente

3.4.2. Certificado de la compañera en Thunderbird

Tras recibir un mensaje firmado por la compañera, su certificado queda almacenado en Thunderbird:

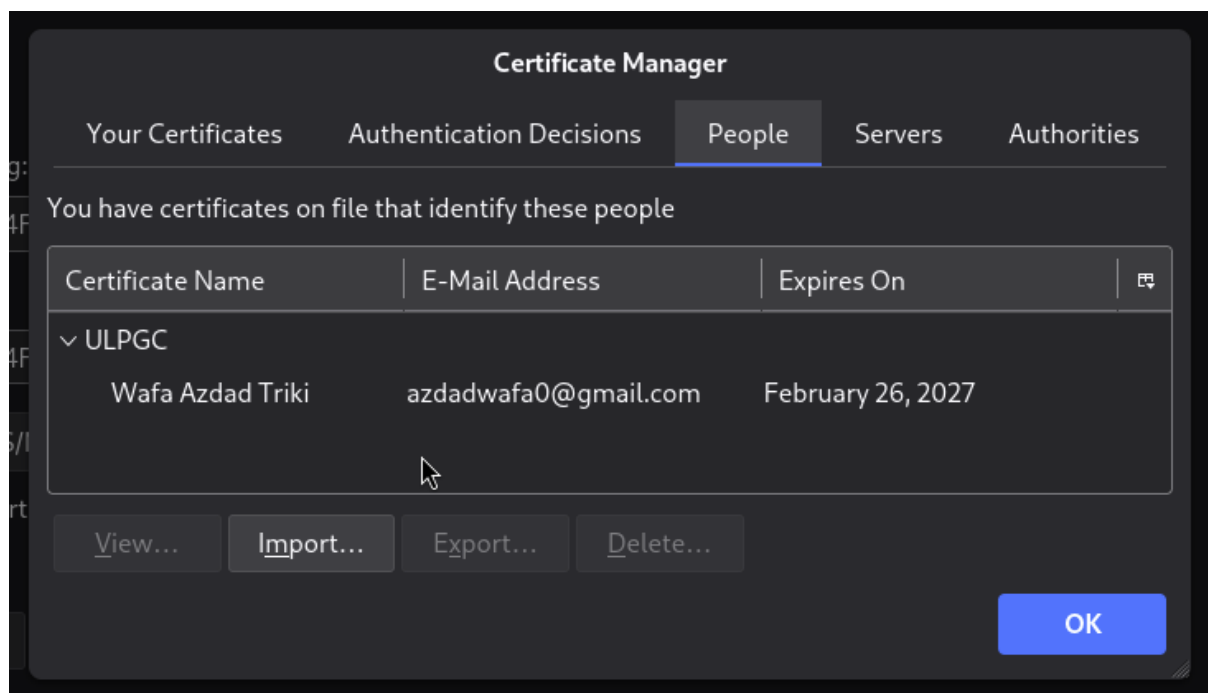


Figura 3.5: Certificado de Wafa Azdad Triki en el almacén de Thunderbird

3.5. Envío de mensaje cifrado y firmado

Una vez que disponemos del certificado de la compañera, podemos enviar un mensaje tanto **firmado** como **cifrado**. El cifrado se realiza con la clave pública del destinatario, de modo que solo él puede descifrarlo con su clave privada.

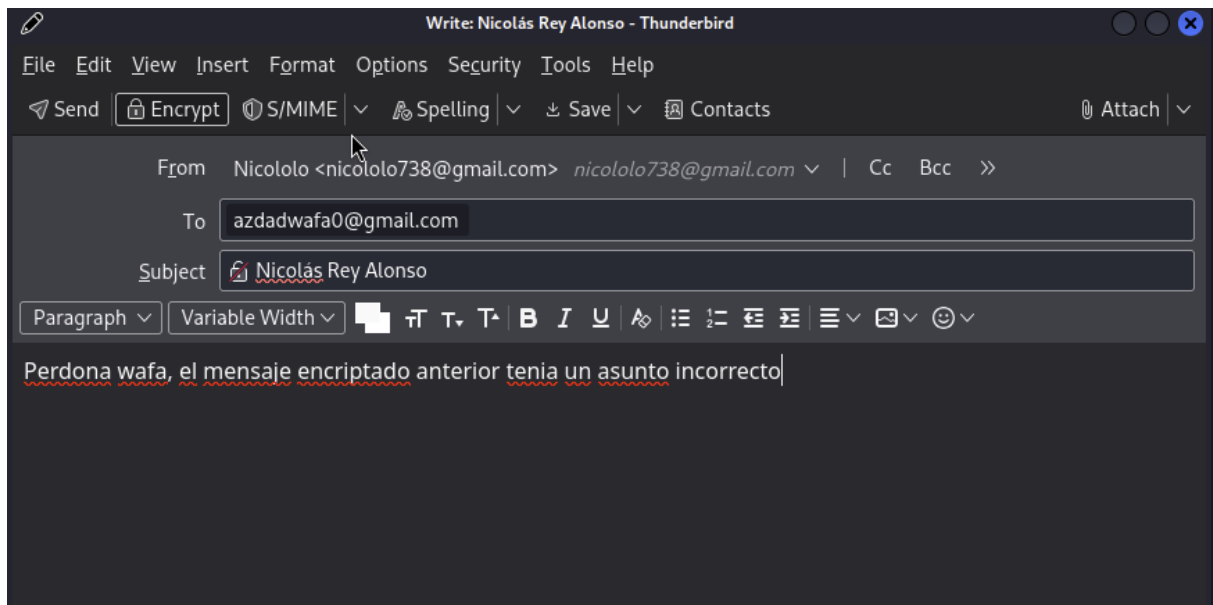


Figura 3.6: Composición de mensaje cifrado y firmado

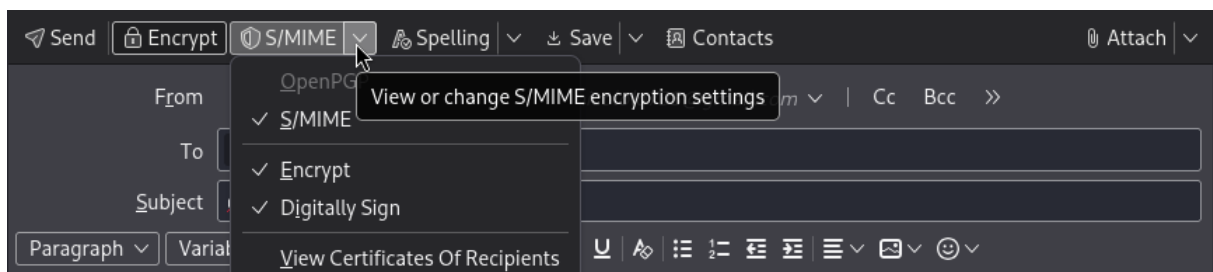


Figura 3.7: Mensaje cifrado y firmado enviado correctamente

3.6. Recepción del mensaje cifrado y firmado de la compañera

Recibimos el mensaje cifrado y firmado de Wafa Azdad Triki en Thunderbird. Se observa el adjunto `smime.p7m` que contiene el mensaje cifrado:

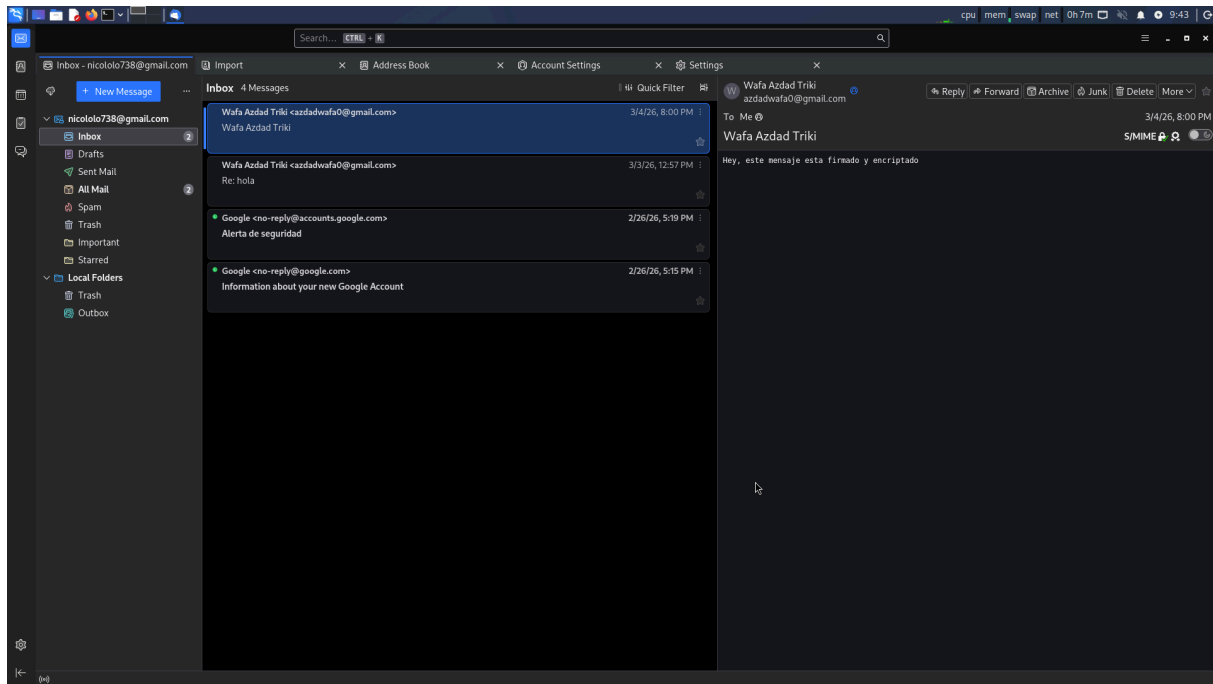


Figura 3.8: Mensaje cifrado y firmado recibido de la compañera

Para el procesamiento con OpenSSL, se guarda el mensaje completo en formato `.eml` (clic derecho → *Guardar como...*) y se exporta el certificado de la compañera en formato `.crt`:

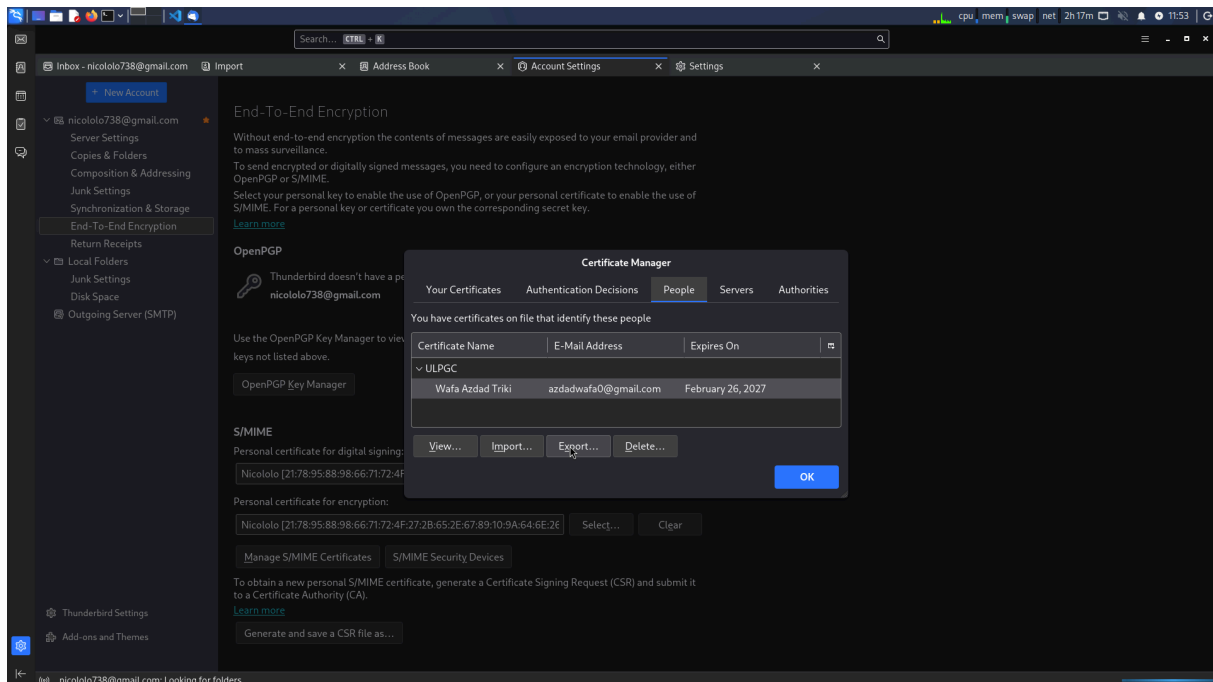


Figura 3.9: Exportación del certificado de la compañera desde Thunderbird

Capítulo 4

Decodificación S/MIME desde Línea de Comandos

4.1. Descripción del proceso

El estándar **S/MIME** (*Secure/Multipurpose Internet Mail Extensions*) permite firmar y cifrar correos electrónicos mediante certificados X.509. Un mensaje que ha sido **cifrado y firmado** contiene una estructura anidada:

1. **Capa externa – Cifrado (SMIME.P7M):** El mensaje está cifrado con la clave pública del destinatario. Contiene un sobre PKCS#7 (`application/pkcs7-mime; smime-type=enveloped-data`).
2. **Capa interna – Firma (SMIME.P7S):** Dentro del mensaje descifrado se encuentra la estructura firmada (`application/pkcs7-mime; smime-type=signed-data`), que contiene el texto original y la firma digital del emisor.

Por tanto, el **descifrado** y la **verificación de la firma** han de realizarse **por separado** y en ese orden:

$$.eml \text{ (P7M cifrado)} \xrightarrow{\text{descifrar}} .eml \text{ (P7S firmado)} \xrightarrow{\text{verificar}} \text{texto plano}$$

4.2. Ficheros necesarios

Para el procesamiento se necesitan tres ficheros:

Cuadro 4.1: Ficheros necesarios para la decodificación S/MIME

Fichero	Descripción
Mensaje .eml	Correo cifrado y firmado, guardado como texto desde Thunderbird
certificadoPersonal.crt + .key	Nuestro certificado y clave privada (para descifrar)
WafaAzdadTriki.crt	Certificado de la compañera (para verificar su firma)

4.3. Paso 1 – Descifrado del mensaje (SMIME.P7M → SMIME.P7S)

Se utiliza `openssl smime -decrypt` con nuestro certificado personal y clave privada, ya que el mensaje fue cifrado con nuestra clave pública:

Listing 4.1: Descifrado del mensaje S/MIME

```
1 openssl smime -decrypt \  
2   -in "mensaje_wafa.eml" \  
3   -recip certificadoPersonal.crt \  
4   -inkey certificadoPersonal.key \  
5   -out mensaje_descifrado.eml
```

Parámetros:

- `-in`: Fichero .eml con el mensaje cifrado (contiene el SMIME.P7M).
- `-recip`: Nuestro certificado personal en formato PEM (.crt), que corresponde a la clave pública con la que se cifró el mensaje.
- `-inkey`: Nuestra clave privada (.key), protegida con contraseña.
- `-out`: Fichero de salida con el contenido descifrado (contiene el SMIME.P7S, es decir, la estructura firmada).

Nota importante: El parámetro `-recip` requiere el certificado en formato PEM (.crt), **no** el fichero PKCS#12 (.p12). Si solo se dispone del .p12, hay que extraer primero el certificado y la clave.

El resultado es un fichero .eml que contiene la cabecera MIME del mensaje firmado, con el adjunto `smime.p7s` visible.

4.4. Paso 2 – Verificación de la firma (SMIME.P7S → texto plano)

Se utiliza `openssl smime -verify` con el certificado de la compañera para comprobar que el mensaje fue firmado por ella y no ha sido alterado:

Listing 4.2: Verificación de la firma S/MIME

```
1 openssl smime -verify \  
2   -in mensaje_descifrado.eml \  
3   -signer WafaAzdadTriki.crt \  
4   -CAfile certificadoRaiz.crt \  
5   -out mensaje_plano.txt
```

Parámetros:

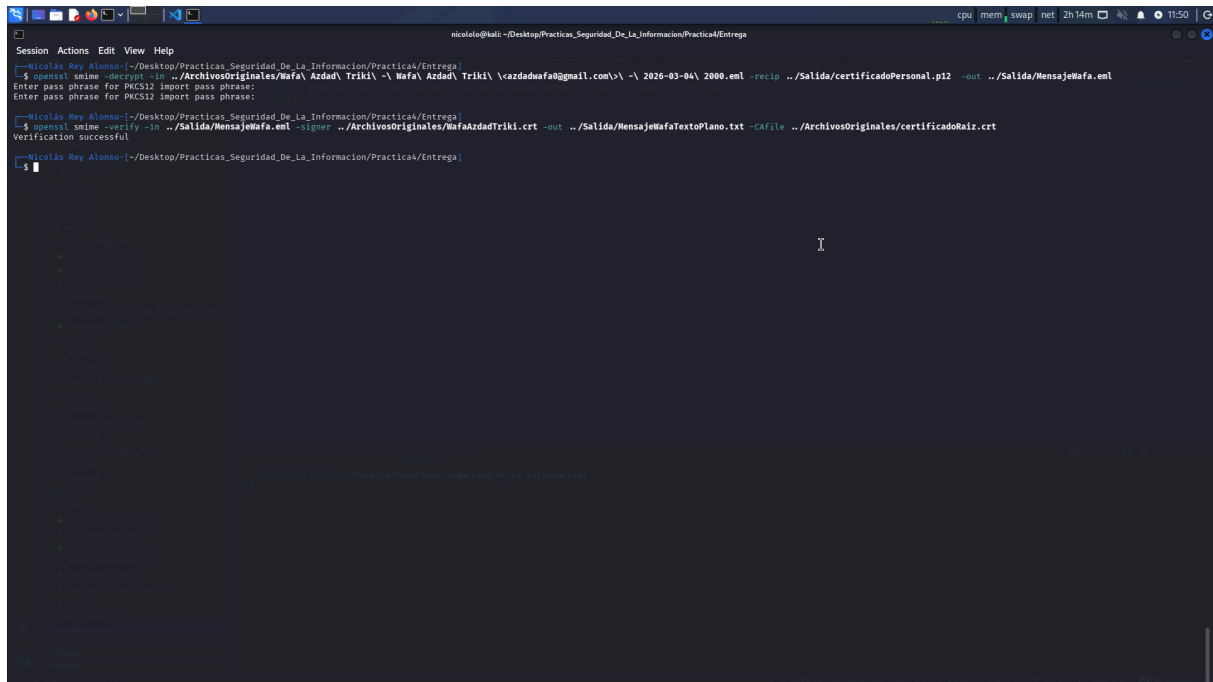
- `-in`: Fichero descifrado que contiene la estructura firmada (SMIME.P7S).
- `-signer`: Certificado del firmante (la compañera) para verificar la firma.
- `-CAfile`: Certificado de la CA raíz. Si ambos usamos la misma CA raíz de la asignatura, la verificación completa funciona. Si la CA es externa o no se dispone de ella, se usa `-noverify` en su lugar.
- `-out`: Fichero de salida con el texto plano del mensaje.

Si el certificado de la compañera fue emitido por una CA diferente, la verificación de cadena fallará con el error `unable to get local issuer certificate`. En ese caso se puede usar el flag `-noverify` para omitir la verificación de la cadena de CA y comprobar únicamente la firma:

Listing 4.3: Verificación sin comprobación de cadena de CA

```
1 openssl smime -verify \  
2   -in mensaje_descifrado.eml \  
3   -signer WafaAzdadTriki.crt \  
4   -noverify \  
5   -out mensaje_plano.txt
```

4.5. Resultado del proceso



```
Session Actions Edit View Help
nicololo@kali: ~/Desktop/Practicas_Seguridad_De_La_Informacion/Practica4/Entrega
[Nicolas Rey Alonso:~/Desktop/Practicas_Seguridad_De_La_Informacion/Practica4/Entrega]
$ openssl smime -decrypt -in ../ArchivosOriginales/Wafa\ Azdad\ Triki\ \caldadwafa@gmail.com>\ -\ 2026-03-04\ 2000.eml -recip ../Salida/certificadoPersonal.p12 -out ../Salida/MensajeWafa.eml
Enter pass phrase for PKCS12:
Enter pass phrase for PKCS12: import pass phrase:
[Nicolas Rey Alonso:~/Desktop/Practicas_Seguridad_De_La_Informacion/Practica4/Entrega]
$ openssl smime -verify -in ../Salida/MensajeWafa.eml -signer ../ArchivosOriginales/WafaAzzadTriki.crt -out ../Salida/MensajeWafaTextoPlano.txt -CAfile ../ArchivosOriginales/certificadoRaiz.crt
Verification successful
[Nicolas Rey Alonso:~/Desktop/Practicas_Seguridad_De_La_Informacion/Practica4/Entrega]
```

Figura 4.1: Proceso completo de descifrado y verificación de firma en terminal

La verificación exitosa muestra el mensaje `Verification successful`, confirmando que:

- El mensaje fue efectivamente firmado por la compañera (autenticación del origen).
- El contenido no ha sido alterado desde su firma (integridad).

El contenido del mensaje descifrado y verificado se muestra a continuación:

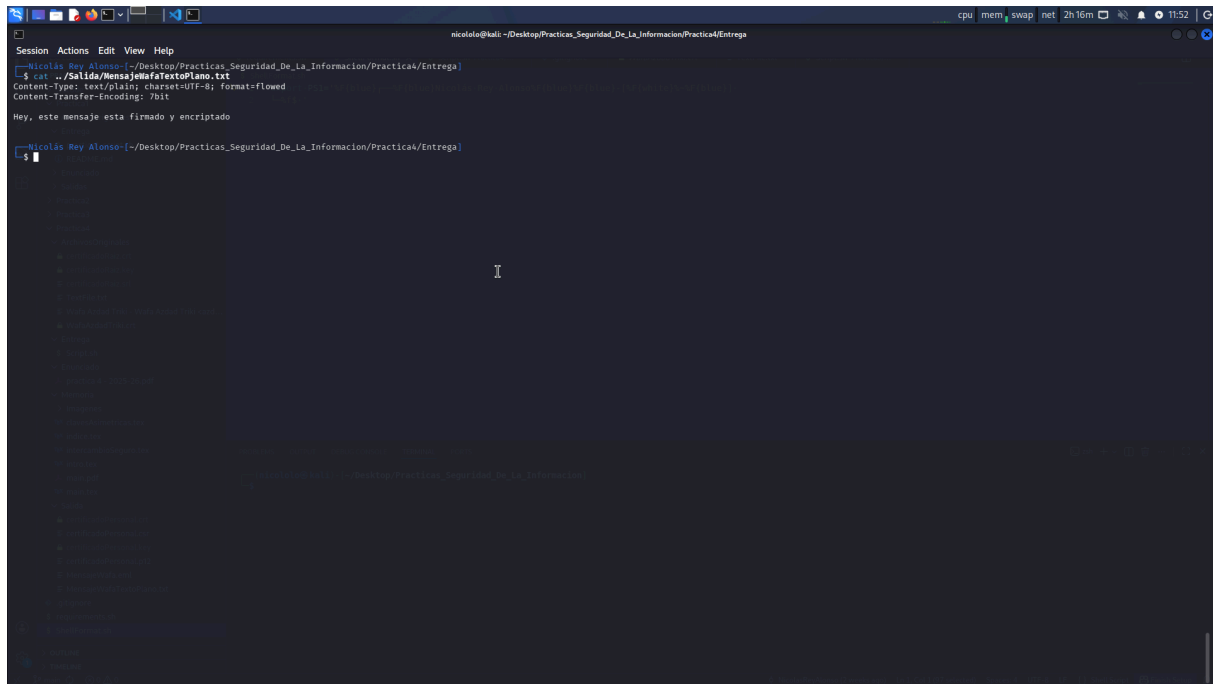


Figura 4.2: Contenido del mensaje descifrado y verificado en texto plano

4.6. Resumen del flujo completo

Cuadro 4.2: Resumen de operaciones S/MIME

Paso	Operación	Comando	Claves utilizadas
1	Descifrar	<code>smime -decrypt</code>	Certificado y clave privada propios
2	Verificar firma	<code>smime -verify</code>	Certificado del firmante (compañera)

Este proceso demuestra el **esquema de doble envoltente** de S/MIME: primero se firma el mensaje (garantizando autenticación e integridad), y luego se cifra el resultado (garantizando confidencialidad). Al recibirlo, se deshacen las capas en orden inverso: primero descifrado, luego verificación de firma.

Capítulo 5

Conclusiones

5.1. Resumen de aprendizajes

A través de esta práctica se han consolidado los siguientes conceptos:

1. **Certificados X.509:** Creación de un certificado personal firmado por una CA raíz, comprendiendo la cadena de confianza y el proceso CSR → firma → certificado.
2. **Formato PKCS#12:** Empaquetado de clave privada y certificado en un único fichero protegido para su importación en aplicaciones.
3. **Configuración de S/MIME en Thunderbird:** Instalación de certificados (raíz en Autoridades, personal en Sus certificados, compañero en Otras personas) y configuración de firma y cifrado.
4. **Intercambio de certificados:** Necesidad de enviar primero mensajes solo firmados para que ambas partes dispongan del certificado del otro antes de poder cifrar.
5. **Esquema de doble envoltente S/MIME:** Firma + cifrado al enviar, descifrado + verificación al recibir, como operaciones separadas.
6. **Decodificación con OpenSSL:** Uso de `openssl smime` para descifrar y verificar mensajes S/MIME desde línea de comandos.

5.2. Conclusiones sobre la práctica

El estándar S/MIME proporciona una solución completa para asegurar comunicaciones por correo electrónico, combinando cifrado (confidencialidad), firma digital (autenticación e integridad) y certificados X.509 (confianza).

Un aspecto fundamental es la gestión de la cadena de confianza: para que la verificación de firma sea completa, ambos interlocutores deben compartir una CA raíz común o disponer de la cadena de certificados intermedia adecuada. En entornos donde esto no es posible, la opción `-noverify` permite comprobar la firma sin validar la cadena de CA.

El formato de los ficheros es crítico: `-recip` de `openssl smime` requiere certificados en formato PEM (`.crt`), no PKCS#12 (`.p12`), lo que obliga a separar clave y certificado.

5.3. Problemas encontrados y soluciones

- **Uso de .p12 con `-recip`:** Inicialmente se intentó usar el fichero `.p12` directamente con `openssl smime -decrypt`, lo que provocaba errores. La solución fue usar los ficheros `.crt` y `.key` por separado.
- **Verificación de cadena de CA:** Al verificar la firma, el error `unable to get local issuer certificate` se resolvió utilizando `-CAfile` con el certificado raíz compartido.
- **Intercambio previo de certificados:** No fue posible cifrar mensajes hasta que ambas partes intercambiaron mensajes firmados, permitiendo que Thunderbird almacenara los certificados.

Bibliografía

- [1] OpenSSL Documentation, <https://www.openssl.org/docs/man3.2/>, 2024.
- [2] Wiki HowTo OpenSSL – CICEI, <https://openssl.cicei.com>, 2025.
- [3] Ramsdell, B., Turner, S., *Secure/Multipurpose Internet Mail Extensions (S/MIME) Version 4.0 Message Specification*, RFC 8551, 2019.
- [4] Cooper, D. et al., *Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile*, RFC 5280, 2008.
- [5] RSA Laboratories, *PKCS #12: Personal Information Exchange Syntax Standard*, v1.1, 2014.
- [6] RSA Laboratories, *PKCS #7: Cryptographic Message Syntax Standard*, RFC 2315, 1998.
- [7] Mozilla, *Thunderbird – Documentación de soporte*, <https://support.mozilla.org/products/thunderbird>.

Apéndice A

Comandos OpenSSL Utilizados

A.1. Creación de certificados

Listing A.1: Comandos para creación del certificado personal

```
1 # Paso b) Generar clave privada RSA con contraseña
2 openssl genpkey -algorithm RSA -out certificadoPersonal.key -aes256
3
4 # Paso c) Crear CSR (solicitud de firma)
5 openssl req -new -key certificadoPersonal.key \
6     -out certificadoPersonal.csr
7
8 # Paso d) Firmar CSR con certificado raiz
9 openssl x509 -req -in certificadoPersonal.csr \
10     -CA certificadoRaiz.crt -CAkey certificadoRaiz.key \
11     -CAcreateserial -out certificadoPersonal.crt -days 365
12
13 # Paso e) Crear fichero PKCS#12
14 openssl pkcs12 -export -in certificadoPersonal.crt \
15     -inkey certificadoPersonal.key \
16     -out certificadoPersonal.p12 -name "Certificado Personal"
```

A.2. Operaciones S/MIME

Listing A.2: Comandos S/MIME para descifrado y verificación

```
1 # Descifrar mensaje S/MIME (P7M -> P7S)
```

```
2 openssl smime -decrypt \  
3     -in mensaje_cifrado.eml \  
4     -recip certificadoPersonal.crt \  
5     -inkey certificadoPersonal.key \  
6     -out mensaje_descifrado.eml  
7  
8 # Verificar firma S/MIME con CA raiz compartida  
9 openssl smime -verify \  
10     -in mensaje_descifrado.eml \  
11     -signer certificadoCompanero.crt \  
12     -CAfile certificadoRaiz.crt \  
13     -out mensaje_plano.txt  
14  
15 # Verificar firma sin verificacion de cadena de CA  
16 openssl smime -verify \  
17     -in mensaje_descifrado.eml \  
18     -signer certificadoCompanero.crt \  
19     -noverify \  
20     -out mensaje_plano.txt
```